

CRUD Meaning

C → Create

R → Read

U → Update

D → Delete

Main Flow

App.jsx

↓

Create.jsx

↓

Display.jsx

Why App.jsx is Main?

Because:

- All data stored there
 - State managed there
 - Parent component controls everything
-

Step 1 → App.jsx

This is the parent component.

Why use useState here?

Because data should be shared to:

- Create component
- Display component

```
const [students, setStudents] = useState([]);
```

Stores all student data.

Step 2 → Pass Data & Functions Using Props

```
<Create addStudent={addStudent} />
```

Why?

Because child component cannot directly access parent functions.

Props send:

- Data
- Functions

from parent → child.

Step 3 → Create.jsx

Purpose:

- Take input from user

Example:

```
<input  
  name="user"  
  value={formData.user}  
  onChange={handleChange}  
/>
```

Why handleChange Used?

Because inputs update dynamically.

```
const handleChange = (e) => {  
  setFormData({  
    ...formData,  
    [e.target.name]: e.target.value  
  });  
};
```

Why Spread Operator ...formData?

Without it:

- old data removed

With it:

- old values remain
 - only changed field updates
-

Step 4 → Form Submit

```
<form onSubmit={handleSubmit}>
```

Why?

To capture form submission.

Step 5 → Add Data

```
addStudent(formdata)
```

Calls parent function.

Step 6 → Parent Receives Data

Inside App.jsx

```
const addStudent = (data) => {  
  setStudents([...students, data]);  
};
```

Why Use Spread Operator?

```
[...students, data]
```

Because:

- React state immutable होता है
- Old array copy करना पड़ता है

Flow:

Old Data + New Data

→ New Array

→ State Update

→ UI Re-render

Step 7 → Display Data

Pass data:

```
<Display students={students} />
```

Step 8 → map() Used

```
students.map((data, index) => (
```

Why?

Because array ko UI me convert karna hai.

READ Operation Complete

User submits form

→ data stored in state

→ data passed to Display

→ map() renders table

DELETE FLOW

Step 1 → Button Click

```
<button onClick={() => deleteStudent(index)}>
```

Why Index Passed?

To identify which row delete karni hai.

Step 2 → Parent Function Call

```
const deleteStudent = (index) => {
```

Step 3 → filter() Used

```
students.filter((_, i) => i !== index)
```

Why filter()?

Because:

- filter returns new array
- selected item remove hoti hai

Flow:

All Data

→ matching index removed

→ new array created

→ state updated

→ UI updated

UPDATE / EDIT FLOW

Step 1 → Click Edit

```
<button onClick={() => editStudent(index)}>
```

Step 2 → Store Index

```
setEditIndex(index)
```

Why?

Because React ko pata hona chahiye:
which row update karni hai.

Step 3 → Store Selected Data

```
setEditData(students[index])
```

Why?

To show old data inside form.

Step 4 → Pass Edit Data to Create.jsx

```
<Create editdata={editdata} />
```

Step 5 → useEffect() Used

```
useEffect(() => {  
  setFormdata(editdata);  
}, [editdata]);
```

Why useEffect?

Because:

- when editdata changes
- form auto-fill hona chahiye

Flow:

Edit button click

→ editdata changes

→ useEffect runs

→ form inputs updated

Step 6 → Submit Updated Data

```
if(editIndex !== null){  
  updateStudent(formdata)  
}
```

Why condition?

Because:

- same form used for Add + Update

Step 7 → Update Array

```
const updatedData = [...students];  
updatedData[editIndex] = data;  
setStudents(updatedData);
```

Why Copy Array First?

React state direct modify nahi karte.

Wrong:

```
students[index] = data
```

Correct:

```
const updatedData = [...students]
```

SEARCH FLOW

Step 1 → Search State

```
const [search, setSearch] = useState("")
```

Step 2 → Input Change

```
onChange={(e)=>setSearch(e.target.value)}
```

Step 3 → Filter Data

```
const filterData = students.filter((data)=>
  data.user.toLowerCase().includes(search.toLowerCase())
)
```

Why toLowerCase()?

For case-insensitive search.

Example:

RAHUL

rahul

Rahul

All match.

Step 4 → Pass Filter Data

```
<Display students={filterData} />
```

Why Not Original Students?

Because:

filtered data display karna hai.

COMPLETE CRUD FLOW

User Input

→ handleChange

→ formData state update

→ handleSubmit

→ add/update function

→ App.jsx state update

→ Display receives props

→ map() renders UI

WHY EACH THING USED?

Thing	Why Used
useState	Store dynamic data
useEffect	Auto update / side effect
Props	Parent → child communication
map()	Display array
filter()	Search/Delete
Spread Operator	Copy old array/object
Conditional Rendering	Show UI based on condition
onChange	Update input
onSubmit	Handle form
index	Identify specific row

MOST IMPORTANT INTERVIEW QUESTION

Explain Your CRUD Project

Answer:

I created a Student CRUD Application using React JS.

Flow:

- User enters data in form
- handleChange updates form state
- onSubmit calls addStudent function
- data stored in students state
- students array passed to Display component
- map() displays table data

For delete:

- filter() removes selected record

For edit:

- selected data stored in edit state
- useEffect prefills form
- updated data replaces old data

For search:

- filter() + includes() used to search records dynamically.